



Génie Logiciel

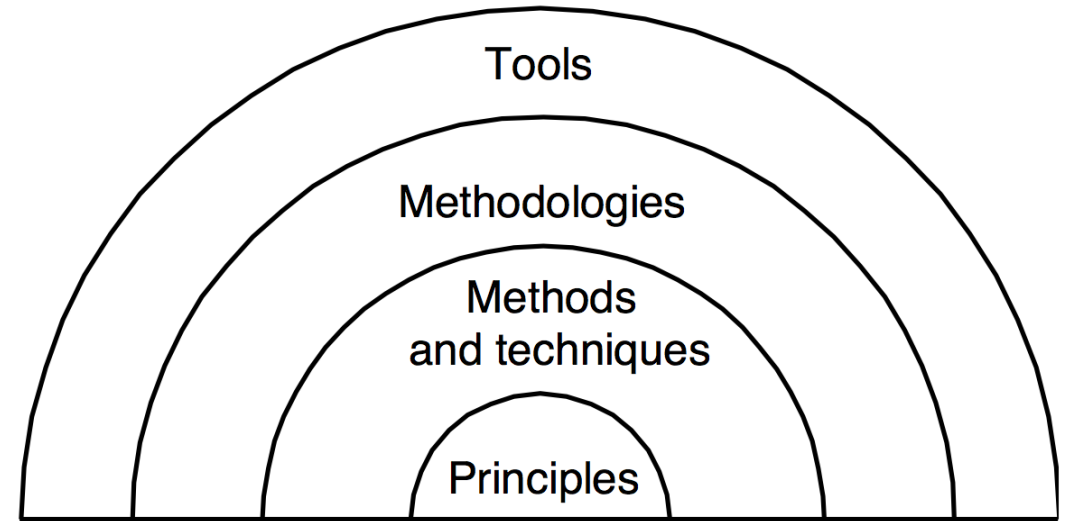
M1 INFORMATIQUE – 2022-2023

Marie NDIAYE

Critères de qualité du logiciel

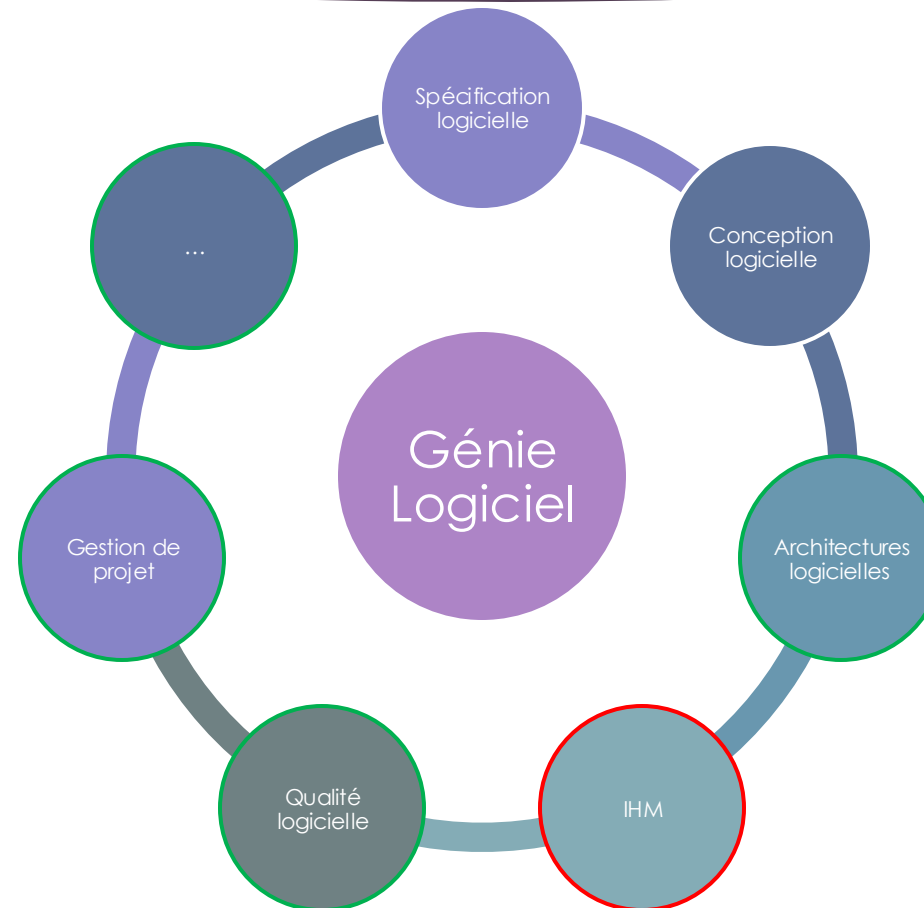
Le génie logiciel

- ▶ Discipline très vaste
- ▶ Liée a tous les domaines de l'informatique
- ▶ Définit des principes de développement
- ▶ Etablit des règles a suivre
- ▶ Invente des techniques
- ▶ Développe des méthodes
- ▶ Propose des outils



Pour garantir la satisfaction de critères établis

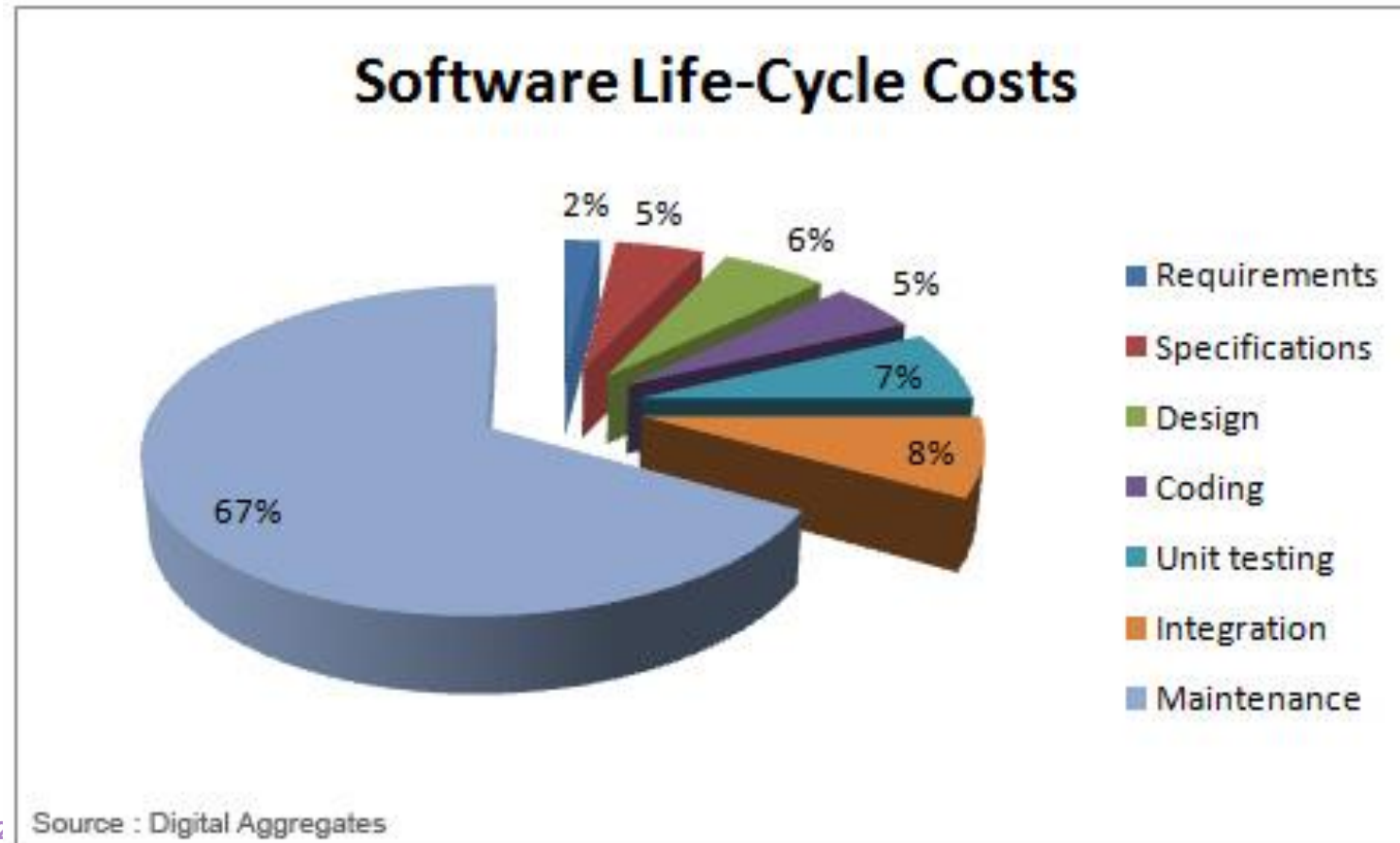
Domaines de connaissance du génie logiciel



Les critères à satisfaire

- ▶ Règle du **CQFD**
 - ▶ Coût
 - ▶ Qualité
 - ▶ Fonctionnalité
 - ▶ Délais

Maîtriser le coût



Fonctionnalité

- ▶ Répondre aux besoins demandés par l'utilisateur, dans l'environnement décrit.
- ▶ Prise en compte dans les critères de qualité.

Respecter les délais

- ▶ Tenir compte
 - ▶ Des tests
 - ▶ Des modifications d'objectifs
 - ▶ De l'intégration
 - ▶ ...

Qualité

- ▶ Ensemble des attributs et caractéristiques d'un produit ou d'un service qui portent sur sa capacité à satisfaire des besoins donnés. (**ANSI**)
- ▶ Aptitude d'un produit ou d'un service à satisfaire les besoins des utilisateurs. (**AFNOR** NF x50-120)
- ▶ La **qualité logicielle** est une appréciation globale d'un logiciel, basée sur de nombreux indicateurs.
- ▶ Elle dépend entièrement de sa construction et des processus utilisés pour son développement.

Qualité : Les normes de l'ISO

► ISO 9126

- Ensemble de normes qui définit le modèle de qualité pour un produit logiciel.
- Concerne la conception, la fabrication et l'utilisation.

► ISO 14 598

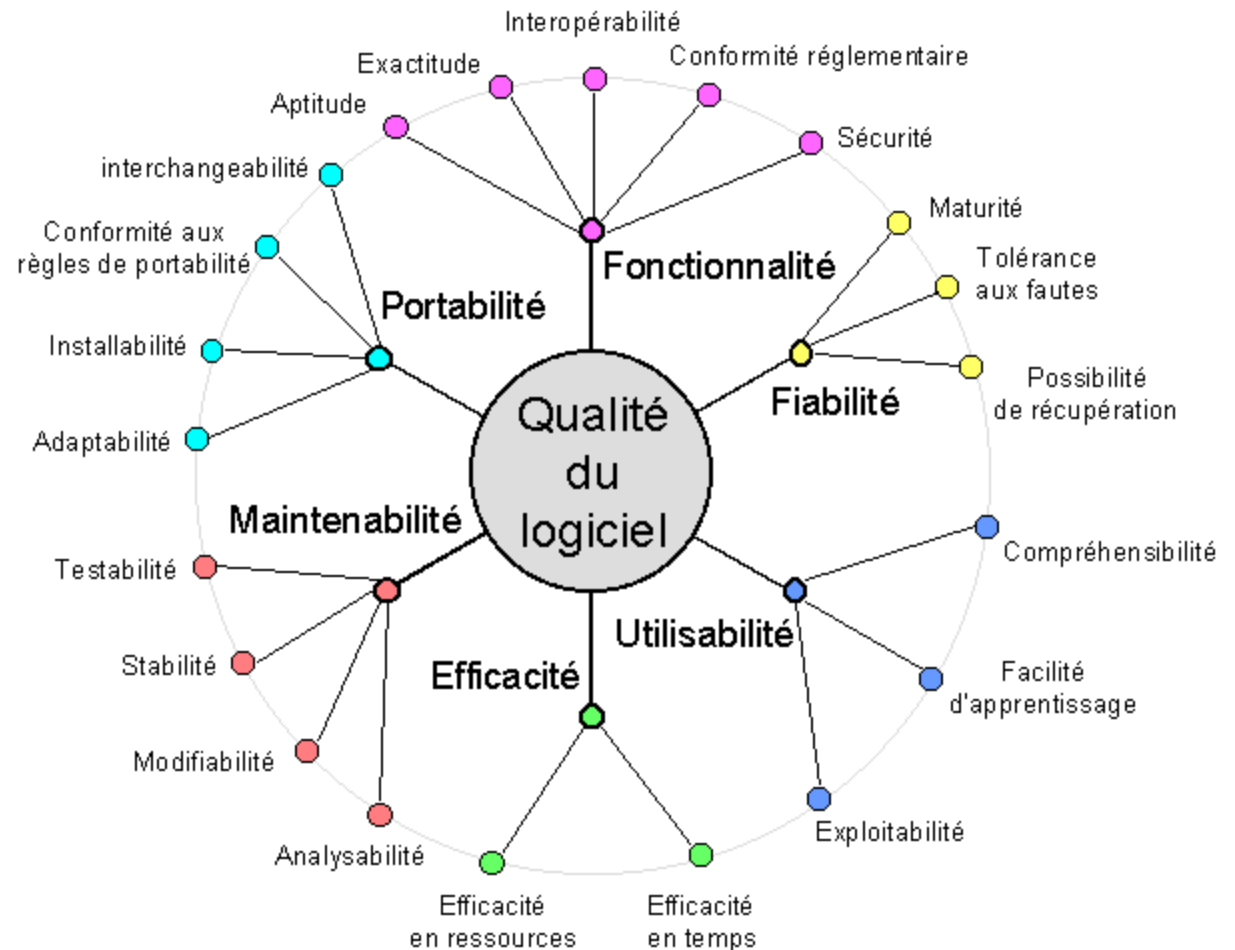
- Ensemble de normes publié par l'AFNOR sous le titre « Ingénierie du logiciel – Évaluation de produit logiciel ».
- Définit les démarches méthodologiques pour l'évaluation de la qualité logiciel.

► ISO 25 000

- Square : Software QUALity Requirements and Evaluation
- Poser le cadre et les références pour définir et évaluer les exigences qualité
- A terme : devrait remplacer les normes ISO 9126 et ISO 14598

La norme ISO/9126

- ▶ Définie par l'ISO
- ▶ Objectif : fournir un cadre pour l'évaluation de la qualité des logiciels
- ▶ Définit six caractéristiques de qualité des produits et fournit des sous caractéristiques.



Fonctionnalité (*Functionality*)

- ▶ Capacité d'un logiciel à fournir les fonctions qui répondent aux besoins formulés et nécessaires quand le logiciel est utilisé dans des conditions spécifiées.
 - ▶ Interopérabilité (*Interoperability*) : capacité à interagir avec un ou plusieurs systèmes.
 - ▶ Aptitude ou adéquation (*Suitability*) : présence d'un ensemble de fonctions appropriées pour les tâches spécifiées.
 - ▶ Exactitude (*Accurateness*) : mise à disposition des résultats ou des effets désirés
 - ▶ Conformité réglementaire (*Compliance*) : le logiciel est conforme aux normes ou conventions liées à l'application ou la réglementation dans les lois et prescriptions similaires.
 - ▶ Sécurité (*Security*) : capacité à empêcher l'accès non autorisé, qu'elle soit accidentelle ou délibérée, à des programmes ou données.

Fiabilité (*Reliability*)

- ▶ Capacité d'un logiciel à maintenir son niveau de service dans des conditions précises pendant une certaine durée.
 - ▶ **Tolérance aux pannes** (*Fault tolerance*) : capacité à maintenir le niveau de performance en cas d'erreur logiciel et de non-respect des interfaces d'interactions avec le logiciel.
 - ▶ **Possibilité de récupération** (*Recoverability*) : facilité de recouvrement en cas défaillance.
 - ▶ **Maturité** (*Maturity*) : capacité à éviter les pannes résultantes d'erreurs dans le logiciel.

Utilisabilité (*Usability*)

- ▶ Capacité d'un logiciel à être compris, appris, utilisé et attrayant pour l'utilisateur quand celui-ci est utilisé dans certaines conditions.
 - ▶ **Compréhensibilité** (*Understandability*) : porte sur l'effort nécessaire des utilisateurs pour reconnaître le concept logique et son applicabilité.
 - ▶ **Facilité d'apprentissage** (*Learnability*) : porte sur l'effort nécessaire pour apprendre son application.
 - ▶ **Exploitabilité** (*Operability*) : porte sur l'effort nécessaire pour effectuer et contrôler des opérations.

Efficacité (*Efficiency*)

- ▶ Capacité d'un logiciel à fournir des performances appropriées, relativement à la quantité de ressources utilisées (moyens matériels, temps, personnes), dans certaines conditions.
 - ▶ **Efficacité en ressources** (*Resource behavior*) : quantité et type de ressources utilisés (CPU, disque, réseau, mémoire, etc.) et la durée associée quand ses fonctions sont invoquées.
 - ▶ **Efficacité en temps** (*Time behaviour*) : capacité à fournir des temps de réponse, de traitement et un débit appropriés quand ses fonctions sont utilisées sous certaines conditions.

Maintenabilité (*Maintainability*)

- ▶ Capacité d'un produit logiciel à être modifié (corrections, améliorations ou des adaptations du logiciel à des changements dans l'environnement, les besoins et les spécifications fonctionnelles)
 - ▶ **Modificabilité** (*Changeability*): capacité à permettre une modification spécifiée d'être implémentée.
 - ▶ **Stabilité** (*Stability*) : capacité à éviter des effets imprévus suite à la modification du logiciel.
 - ▶ **Testabilité** (*Testability*) : capacité à être validé par rapport à une spécification.
 - ▶ **Analysabilité** (*Analyzability*): effort nécessaire pour le diagnostic des défaillances et des causes des échecs, ou pour l'identification des pièces à modifier.

Portabilité (*Portability*)

- ▶ Capacité d'un produit logiciel à être transféré d'un environnement (organisationnel, matériel ou logiciel) à un autre
 - ▶ **Adaptabilité** (*Adaptability*) : capacité à s'adapter à différents environnements spécifiés sans appliquer d'autres actions ou moyens que ceux prévus à cet effet.
 - ▶ **Installabilité** (*Installability*) : effort nécessaire pour installer le logiciel dans un environnement spécifié.
 - ▶ **Conformité aux règles de portabilité** (*Conformance*) : capacité du logiciel à être conforme aux normes et conventions relatives à la portabilité.
 - ▶ **Interchangeabilité** (*Replaceability*) : capacité à être utilisé à la place d'un autre logiciel spécifié dans l'environnement de celui-ci.

Comment assurer la qualité du logiciel ?

- ▶ Accorder une attention particulière aux phases d'analyse et de conception
 - ▶ Rédiger des spécifications fonctionnelles détaillées et claires.
 - ▶ Impliquer les parties prenantes dès le début pour s'assurer que les besoins sont bien compris.
- ▶ Adopter des modèles de cycle de vie ou des méthodologies de développement
- ▶ Effectuer des revues régulières du code par les pairs
- ▶ Effectuer les tests unitaires, d'intégration, fonctionnels, de performance, etc.
- ▶ Mettre en place un dispositif d'intégration continue pour détecter rapidement les problèmes lors de l'ajout de nouvelles fonctionnalités.
- ▶ Recueillir les retours des utilisateurs pour identifier les problèmes et améliorer l'expérience.
- ▶ Maintenir la documentation à jour
- ▶ Utiliser des outils pour l'analyse de code, les tests, la gestion de projet, le versioning, la documentation, etc.
- ▶ Former l'équipe aux meilleures pratiques de qualité logicielle et les tenir informés des nouvelles tendances et outils.
- ▶ S'appuyer sur des procédés et des outils techniques d'assurance qualité.

Références

- ▶ Alain April et Claude Laporte, Assurance qualité logicielle 1: concepts de base, Lavoisier, 2011, (ISBN 9782746231474), page 387
- ▶ Jean Menthonnex - CERSSI, Sécurité et qualité informatiques: nouvelles orientations, PPUR presses polytechniques - 1995, (ISBN 9782880742881), page 77
- ▶ <http://www.cse.dcu.ie/essiscope/sm2/9126ref.html>
- ▶ <http://www.yves-constantinidis.com/doc/yves-758.htm>
- ▶ http://fr.wikipedia.org/wiki/Qualit%C3%A9_logicielle
- ▶ <http://www.uti-informations.com/repupload/upload-extranet/siteadmin/smq/iso9126.pdf>
- ▶ <http://www.redsen-consulting.com/2011/01/quest-ce-que-la-qualite-logicielle/>
- ▶ <http://lim.univ-reunion.fr/staff/courdier/>